АННОТАЦИИ

java

Тачилина М. А.
5030102/40004

ПРОБЛЕМА

До аннотаций:

- конфигурации в XML
- маркерные интерфейсы
- внешние файлы

ЧТО ТАКОЕ АННОТАЦИИ?

Мета-аннотации
(@Target, @Retention, @Documented, @Inherited, @Repeatable)

Аннотации в языке программирования Java представляют специальную форму метаданных (данных о данных), которую можно добавлять в исходный код. Они предоставляют дополнительную информацию о программе, которая может использоваться компилятором, инструментами сборки, фреймворками или во время выполнения.

ПРИНЦИПЫ ОБРАБОТКИ АННОТАЦИЙ В КОМПИЛЯЦИИ

СЛУЖАТ ТРИГЕРРАМИ

для активации обработчиков аннотаций, которые выполняют генерацию дополнительных исходных файлов

МЕХАНИЗМ ОБРАБОТКИ

является циклическим. Итерации продолжаются до тех пор, пока в результате очередного раунда не перестанут появляться новые файлы

ТИП ХРАНЕНИЯ (RETENTION)

1 SOURCE

2 CLASS

3 RUNTIME

ТИП ОБЪЕКТА, НАД КОТОРЫМ ОНА УКАЗЫВАЕТСЯ (TARGET)

ANNOTATION_TYPE

FIELD

METHOD

PARAMETER

LOCAL_VARIABLE

TYPE

PACKAGE

CONSTRUCTOR

ПРИМЕНЕНИЕ АННОТАЦИЙ

АННОТАЦИИ ОБЪЯВЛЕНИЙ

«классические» аннотации, которые ставятся непосредственно перед объявлением какой-либо программной единицы

АННОТАЦИИ ИСПОЛЬЗОВАНИЯ ТИПОВ

аннотируют не переменную/класс/метод, а конкретное использование этого типа в программе

АННОТАЦИИ ОБЪЯВЛЕНИЙ

8

```
@NotNull
private List<String> список;
```

```
private List<@NotNull String> список;
```

АННОТАЦИИ ИСПОЛЬЗОВАНИЯ ТИПОВ

ПОЛЬЗОВАТЕЛЬСКИЕ АННОТАЦИИ

Виды

- маркерная;
- с одним значением;
- с несколькими значениями

Синтаксис

Для объявления аннотации используется ключевое слово @interface.

Например:

```
public @interface Description { String title(); int version() default 1; String text(); }
```

ВСТРОЕННЫЕ АННОТАЦИИ

Имя	Значение
@Override	указывает компилятору, что текущий метод переопределяет метод из суперкласса
@Deprecated	эта аннотация помечает элемент (класс, метод, поле) как устаревший
@SuppressWarnings	отключает определенные предупреждения компилятора для аннотированного элемента
@FunctionalInterface	указывает, что интерфейс предназначен для использования в качестве функционального интерфейса (т.е. он должен иметь ровно один абстрактный метод)

@OVERRIDE

Retention: SOURCE;
Target: METHOD.

ЗАЧЕМ?

1) Верификация намерений программиста (Самопроверка)

2) Документирование и читаемость кода

3) Условия корректного переопределения:

- сигнатура метода;
- тип возвращаемого значения;
- модификатор доступа;
- нестатические методы;

```
class Parent {
    public void setName(String name) {
        System.out.println("Устанавливаю имя в родителе");
    }
}

class Child extends Parent {
    // Я ДУМАЮ, что переопределяю метод родителя
    public void setName(String name, int age) {
        System.out.println("Устанавливаю имя и возраст");
    }
}
```

СПАСИБО ЗА ВНИМАНИЕ!